

FIAP · MBA AI ENGINEERING & MULTI-AGENTS

CLOUD & COGNITIVE ENVIRONMENTS

Criando Agentes com Azure Foundry AI

Consolidação do curso — conceitos, a plataforma, a escolha do modelo, o custo, a estratégia de alternância e o laboratório do Deva contínuo.

PROF. ISAIAS S. BRITTO · SETEMBRO DE 2026

O caminho, e o que cada trecho decide

Cada bloco responde uma pergunta que aparece, nesta ordem, em todo projeto de agente que chega à produção

1 · CONCEITOS

O que é, de fato, um agente

Loop, contexto, memória, ferramentas, autonomia e modelo. Seis camadas — e a última é a única que todo mundo copia.

2 · A PLATAFORMA

O que o Foundry resolve

Hierarquia, capacidades prontas, os três tipos de agente e as três setas de governança que saem do desenho.

3 · O MODELO

Por qual porta ele entra

OpenAI, Anthropic, Kimi, DeepSeek e Gemini — e por que a porta de entrada muda cobrança, crédito, cota e SLA.

4 · CUSTO

Onde nasce e o que freia

Oito medidores, a conta de um agente real, as armadilhas e os três painéis que não batem — de propósito.

5 · ESTRATÉGIA

Quando usar cada um, e quando alternar

A árvore de quatro perguntas, Foundry × Container Apps × Functions e os gatilhos observáveis de troca.

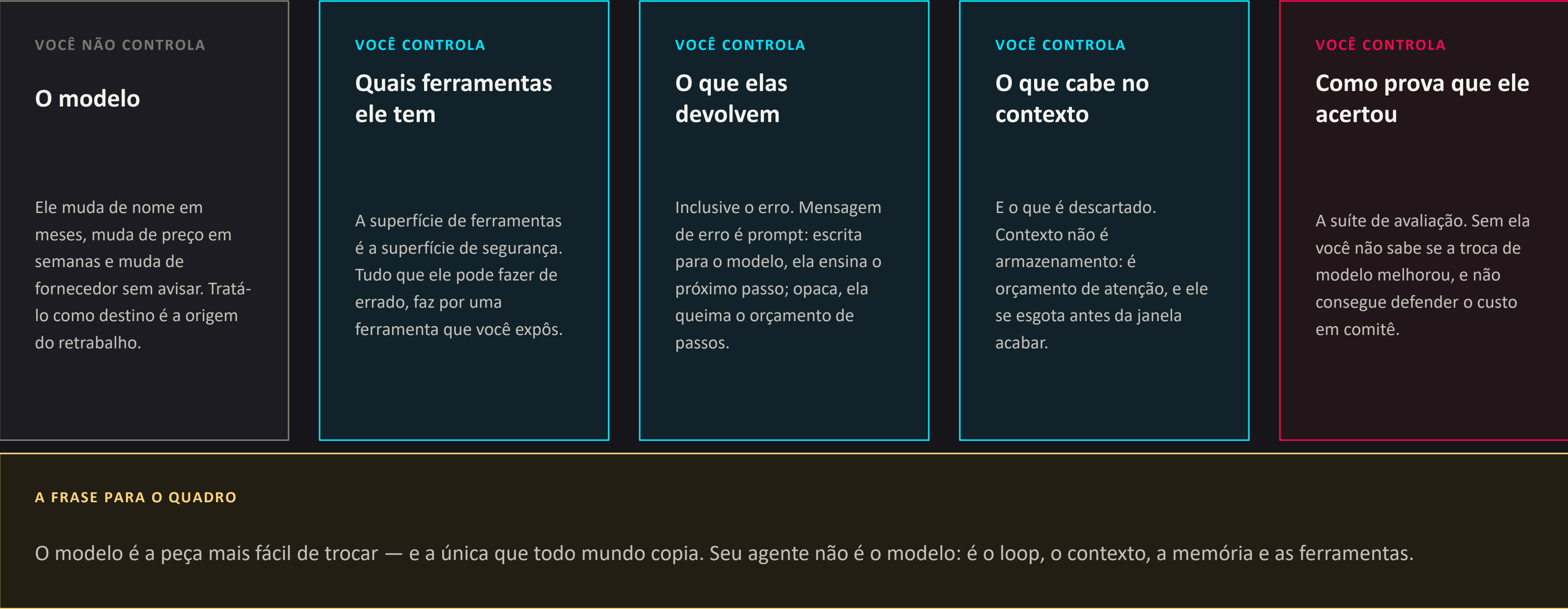
6 · LABORATÓRIO

O Deva contínuo, no repositório

Memória revisável, fila de exceções e a fronteira em código: 14 operações na API, 5 na mão do agente.

O trabalho migrou do prompt para o ambiente

É a tese que amarra os seis blocos — e o critério para julgar qualquer plataforma do mercado



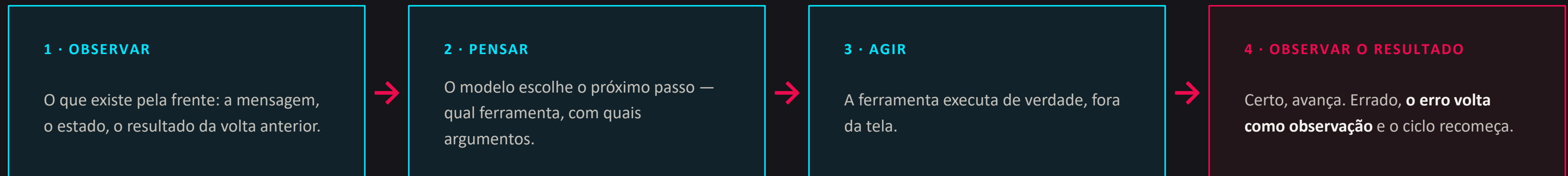
1

O que é um agente

Ele não responde: persegue um objetivo até acabar o trabalho — ou o orçamento.
Seis camadas, e cada uma resolve o que a anterior deixou aberto.

O agente loop — e os três freios

A seta que volta é o agente; o freio é o que o torna pagável



```
for volta in range(MAX_VOLTAS):          # freio 1 · iterações
    r = modelo.responder(mensagens, ferramentas=CATALOGO)
    if not r.tool_calls:
        return r.texto                    # ele decidiu que terminou
    for chamada in r.tool_calls:
        try:    saida, erro = executar(chamada), False
        except Exception as e:
            saida, erro = f"ERRO: {e}", True      # o erro VOLTA
    mensagens.append(resultado(chamada, saida, erro))
```

Freio 1 · voltas — `max_voltas = 8`
Um erro que se repete vira laço infinito.

Freio 2 · orçamento — `20 mil tokens`
É o freio que aparece na fatura.

Freio 3 · tempo — `timeout = 60 s`
Evita o travamento silencioso.

O difícil não é escrever o laço. É **saber onde ele deve parar** — e escrever o limite em código, não no prompt.

Chatbot × agente × harness

A turma mistura as três — e é a mistura que faz comprar a plataforma errada

CHATBOT

Pergunta → resposta → fim

Nenhuma ferramenta é chamada.
Nada acontece fora da tela. A
conversa seguinte começa do zero.

Custo previsível, valor limitado a
quando você para de digitar.

AGENTE

**Objetivo → N passos →
entrega**

Você dá um objetivo, ele decide os
passos. Chama ferramentas de
verdade: lê, calcula, escreve, integra.
Carrega contexto e memória entre
sessões.

Custo variável — por isso precisa de
freio.

HARNESS

Onde o loop roda

Claude Code no terminal · OpenClaw
e Hermes como runtime autônomo ·
Foundry Agent Service como serviço
gerenciado.

Erro comum: achar que é opcional.

O QUE NÃO MUDA

**Frequentemente é o
mesmo modelo**

A diferença entre um chatbot e um
agente quase nunca é o modelo. É o
laço e o acesso a ferramentas.

O modelo é o motor; o harness é o
carro. Trocar um não obriga a trocar
o outro — e é isso que dá
portabilidade.

Construa no de sessão · opere no gerenciado · só vá para o persistente com isolamento e orçamento de verdade.

As seis camadas de um agente

Cada uma resolve um problema que a anterior deixou aberto — e a última é a mais fácil de trocar

	CAMADA	O PROBLEMA QUE RESOLVE	O ARTEFATO	NO MICROSOFT FOUNDRY
01	Loop	Responder não é trabalhar. O agente persegue um objetivo.	o laço + os três freios	o serviço executa o laço no agente prompt
02	Contexto	Ele age, mas não conhece você, o projeto nem a forma de trabalho.	AGENTS.md — até 200 linhas	campo Instruções , versionado a cada publicação
03	Memória	Contexto diz quem você é; não faz lembrar do que foi ensinado ontem.	MEMORY.md — regra, data e dono	Memory (preview): perfil, resumo e procedural
04	Ferramentas	Agente sem ferramenta decide bem e não faz nada.	MCP · SKILL.md · nativas	catálogo, OpenAPI, MCP, A2A, Skills (preview)
05	Autonomia	Onde ele mora, quanto tempo fica ligado, quanto risco carrega.	sessão · hospedado · persistente	identidade própria no Entra ID, escala a zero
06	Modelo	Todo agente pensa com alguma coisa. E ela tem preço.	uma constante, nunca uma string espalhada	catálogo com modelos vendidos pela Azure e de parceiros

As cinco primeiras camadas são **trabalho seu** e não se copiam. A sexta é a única que todo mundo copia — e é a que as pessoas passam mais tempo discutindo.

O AGENTS.md e a regra das 200 linhas

Contexto não é «mais informação». É a informação que muda a decisão

O TESTE DE ADMISSÃO DE CADA LINHA

«Se eu apagar esta linha, o agente decidiria diferente?»

Se a resposta for «não», a linha sai. É assim que se cumpre a regra das 200 linhas — e é assim que o arquivo continua legível no sexto mês.

RUIM — NÃO MUDA NENHUMA DECISÃO

«O Deva é um assistente inteligente que ajuda a XPTO de forma eficiente.»

Bonito, verdadeiro e inútil: nenhuma decisão do agente muda por causa dessa frase.

BOM — MUDA TODAS

«Na dúvida eu REVISO, nunca aprovo: falso positivo custa mais que fricção.»

É uma regra de decisão, com o motivo junto.

AS ONZE SEÇÕES

- 1 · Identidade e escopo — quem é, para quem trabalha, quem valida
- 2 · Definição de pronto — os campos obrigatórios e a regra de fallback
- 3 · Fluxo de trabalho — a ordem dos passos, começando por LER A MEMÓRIA
- 4 · Fontes de verdade — a precedência: política > extração > memória > julgamento
- 5 · Listas fechadas — categorias enumeradas. «Nunca invente categoria»
- 6 · O que nunca faz sem humano — aprovar, lançar, comunicar, alterar a política
- 7 · Regras de memória — quem escreve, com origem e data
- 8 · Ferramentas autorizadas — cada uma com o seu limite de uso
- 9 · Orçamento e parada — passos, tentativas, custo, timeout, com NÚMEROS
- 10 · Formato de saída — o JSON, com um exemplo preenchido
- 11 · Tom — objetivo, PT-BR, incerteza declarada

O que NÃO entra: biografia da empresa, história do setor, texto motivacional — qualquer parágrafo que não complete a frase «...logo, o agente deve fazer X em vez de Y».

Contexto, thread e memória são três coisas

Contexto diz quem você é. Thread guarda a conversa de hoje. Memória é o que sobrou de aprendizado depois que ela acabou

VIVE UMA CHAMADA

Contexto

«Você é o Deva, responde em JSON.» Vai junto em toda requisição e some com ela.

VIVE UMA SESSÃO

Conversa / thread

As doze mensagens de hoje. No Foundry, gerenciada pelo serviço — é o nível 1 da continuidade.

VIVE PARA SEMPRE

Memória

«O limiar de aprovação é 0,80, definido pelo risco.» Sobrevive à sessão, e por isso precisa de dono e de data.

COMO SE PREENCHE — UMA LINHA POR REGRA

```
## Parâmetros de decisão      <- lidos em tempo de execução
- [2026-08-30] limiar_aprovacao = 0.80 (definido por: risco)
- [2026-08-30] max_rostos = 1 (definido por: risco)

## Regras aprendidas
- [2026-08-30] Foto contra a janela costuma cair na revisão
  sem que a pessoa esteja errada; orientar nova captura em
  vez de recusar. (definido por: operação)
```

POR QUE O ARQUIVO

Muda sem redeploy — editar o markdown muda o comportamento na próxima requisição.

Tem data e dono — toda regra carrega quando entrou e quem decidiu. É auditoria de graça.

O limiar é do negócio — quem define 0,80 é a área de risco; o desenvolvedor implementa a leitura.

O QUE NUNCA ENTRA

Dado pessoal, caso individual, nome de pessoa física, hipótese não confirmada — e, em projeto biométrico, imagem, coordenada de rosto ou template.

Memória guarda REGRA, nunca PESSOA. Regra generaliza; caso envelhece e vira passivo.

Dado biométrico é sensível — LGPD, art. 11.

Três coisas diferentes que a turma confunde

Agente sem ferramenta decide bem e não faz nada. Ferramenta é o que o tira da tela

A TOMADA

MCP

Protocolo padrão para conectar o agente a sistemas. Criado pela Anthropic e doado à Agentic AI Foundation (Linux Foundation) em 09/12/2025.

A spec 2026-07-28 tornou o protocolo stateless — pronto para produção atrás de balanceador.

No portal do Foundry, MCP é literalmente uma opção de menu.

O MANUAL

Skills

Procedimento salvo em Markdown, ensinado uma vez. Padrão aberto (agentskills.io), com progressive disclosure em três estágios: nome e descrição sempre; o corpo do SKILL.md sob demanda; anexos só se a skill pedir.

É isto que permite ter 50 skills sem estourar o contexto — e por isso DESCRIÇÃO VAGA = SKILL QUE NUNCA RODA.

AS MÃOS

Ferramentas nativas

O que o harness já sabe fazer. Na Foundry: Intérprete de código, Pesquisa de arquivos, Azure AI Search, OpenAPI, Uso do Computador e A2A.

No laboratório usamos OpenAPI: a ferramenta do agente é a API que VOCÊ escreveu. Ferramenta não se baixa — se escreve.

Skill não executa nada. Ela instrui o agente a usar as ferramentas na ordem certa. Quem executa é a ferramenta; quem conecta ao sistema externo é o MCP.

Sessão · hospedado · persistente

O roteiro fala de sessão e persistente. Falta a do meio — onde as empresas realmente operam

	ONDE RODA	EXEMPLO	CUSTO PARADO	SUPERFÍCIE DE RISCO	BOM PARA
Sessão	máquina do dev	Claude Code	zero	a máquina do dev	construir e testar
Hospedado	nuvem, sob demanda	Foundry · Container Apps	≈ zero (escala a zero)	o escopo da identidade	OPERAR
Persistente	máquina ligada 24×7	OpenClaw · Hermes	contínuo	a máquina inteira	operação pessoal

O RISCO DA AUTONOMIA, EM CONTROLES

- Custo fora de controle — loop sem parada, retry infinito → max_voltas e orçamento com alerta
- Prompt injection — instrução escondida em documento ou imagem → conteúdo lido é DADO, nunca instrução
- Skill ou MCP malicioso — pacote de terceiro com acesso total → curadoria, revisão de código, escopo mínimo
- Excesso de permissão — agente com chave de admin → identidade gerenciada + RBAC mínimo
- Vazamento de dado — grava sensível em log ou memória → política no AGENTS.md, log sem payload

O INCIDENTE QUE ACONTECEU DE VERDADE

Uma nota fiscal com instrução no rodapé

Na Linha 2, chegou uma nota com uma frase escondida mandando aprovar sem revisão. O Deva RECUSOU e registrou como incidente — exatamente como a seção «Limites» do AGENTS.md mandava.

Ele acertou. Mas repare no que teria acontecido se errasse UMA vez, num agente que escreve a própria memória: a frase viraria regra permanente, aprovada por ele mesmo, aplicada a todos os documentos seguintes. Ninguém perceberia até a auditoria externa.

Aprendizado automático sem revisão não é funcionalidade. É superfície de ataque — e a defesa não é escrever «não faça isso». É não expor a operação.

2

O Foundry

Um agente que funciona não é um agente que a empresa aceita.
A diferença entre os dois é este bloco inteiro.

A definição que vale

O Foundry não é um chat com modelo. É onde o agente ganha endereço, identidade, conta e histórico

ANTES DO FOUNDRY

Você tem um script que chama uma API

Quem rodou? a sua credencial

Quanto custou? ninguém sabe

O que ele fez? o que estiver no seu log

Como desligo? apagando o seu código

Voltar atrás? refazer na mão



DEPOIS DO FOUNDRY

Você tem um agente com endereço próprio

Quem rodou? o principal DELE no Entra ID

Quanto custou? está no painel, por projeto

O que ele fez? rastreamento passo a passo

Como desligo? revoga a identidade

Voltar atrás? versão anterior do agente

POR QUE ISSO É ASSUNTO DE MBA

Nenhum comitê de risco aprova um agente porque ele responde bem.

Aprova porque consegue responder três perguntas: **o que ele fez, quanto custou e como eu desligo.**

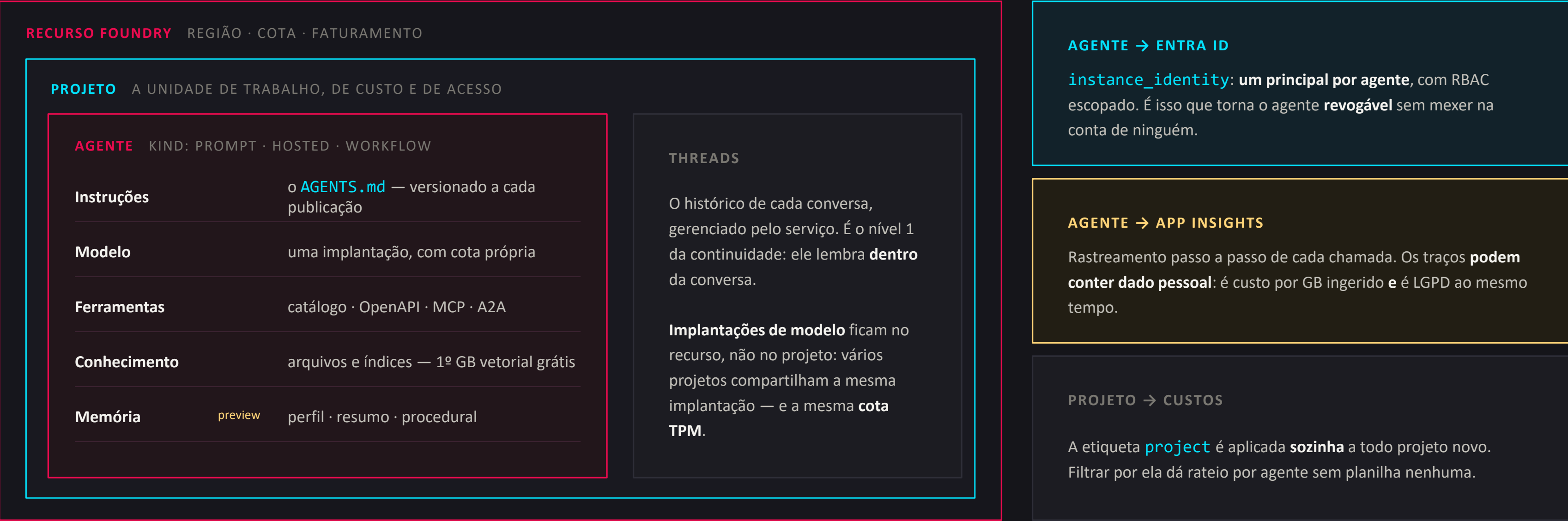
E é por isso que a decisão de plataforma quase nunca é sobre capacidade do modelo. Ela é sobre **dado e residência, identidade corporativa, sandbox de execução e plano de controle** — os quatro motivadores.

Se nenhum deles aparece como requisito, você ainda não precisa da plataforma.

Marque a mudança de nome: o **Azure AI Foundry** passou a se chamar **Microsoft Foundry**. Documentação antiga usa o nome antigo; o portal usa o novo.

A hierarquia que explica todo o resto

Recurso → projeto → agente. Região e cota moram no recurso; agentes, ferramentas e custo, no projeto



O que vem pronto no Foundry

Seis capacidades — e a nota cinza de cada cartão é o que você teria de construir e manter sozinho

MODELOS Catálogo com modelos vendidos pela Azure e de parceiros, atrás de uma API só. Implantação, cota e versão por modelo. sem isso: uma chave e um SDK por fornecedor	FERRAMENTAS Catálogo pronto, OpenAPI para a sua API, MCP para apps de terceiros e A2A para outro agente. sem isso: você escreve o adaptador de cada uma	CONHECIMENTO Arquivos e índices ligados ao agente, com armazenamento vetorial gerenciado — o primeiro 1 GB é gratuito. sem isso: pipeline de ingestão e índice próprios
MEMÓRIA (PREVIEW) Perfil do usuário, resumo de conversa e procedural. Ciclo de extração, consolidação e recuperação. limites: 100 escopos · 10 mil memórias por escopo · 1.000 req/min · sem VNet	IDENTIDADE Um principal por agente no Entra ID (instance_identity), com RBAC escopado. é isto que torna o agente revogável sem mexer na conta de ninguém	OBSERVABILIDADE Rastreamento passo a passo no Application Insights, avaliações e a etiqueta project aplicada sozinha. rateio por projeto sem planilha — e traço é custo e é LGPD ao mesmo tempo

Nada disso deixa o agente mais inteligente. Tudo isso deixa o agente **auditável, orçado e revogável** — que é o que um comitê de risco aprova.

Os três tipos de agente

Três contas diferentes — e uma regra que economiza semestre: ninguém começa em hosted

CRITÉRIO	PROMPT · COMECE AQUI	HOSTED	WORKFLOW
O que é	Agente definido por instrução + ferramentas do catálogo. Sem código.	O seu código Python ou C# em contêiner gerenciado pelo Foundry.	Coordena outros agentes: chama, espera, decide o próximo.
Quando é a resposta certa	A tarefa é responder com base em conhecimento e ferramentas prontas.	Framework próprio, biblioteca específica, estado em disco ou outro protocolo.	Vários papéis distintos com regras de passagem entre eles.
Cobrança do agente	Nenhuma — você paga modelo e ferramentas.	vCPU-hora + GiB-hora, além de modelo e ferramentas.	Nenhuma pela orquestração.
Tempo até o 1º resultado	Minutos, pelo portal.	Horas ou dias: repositório, imagem, implantação, credenciais.	Depende dos agentes que ele coordena.
Estado	Threads gerenciados pelo serviço.	\$HOME até 20 GiB por sessão; sessão viva por até 30 dias; escala a zero.	Estado do fluxo, gerenciado pelo serviço.
Identidade	Principal próprio no Entra ID.	Principal próprio no Entra ID.	Herda dos agentes coordenados.
Risco típico	Instrução vaga: ele responde o que não devia.	Contêiner ligado sem uso e imagem sem manutenção.	Laço entre agentes e custo multiplicado sem ninguém perceber.
Sinal de promover	—	Você já escreveu três gambiarras na instrução para simular o que uma função resolveria.	Um único agente já tem instruções contraditórias para papéis diferentes.

O erro mais caro é **escolher a opção complexa primeiro**. Um agente **hosted** que poderia ser **prompt** custa infraestrutura, tempo de equipe e manutenção de imagem — para entregar o mesmo resultado.

Para quem é a entrega?

A pergunta que reposiciona o projeto — e que muda quem aprova o orçamento

A PROVOCAÇÃO

«Se este agente parar de funcionar amanhã, quem liga reclamando?»

Se a resposta é o **time de tecnologia**, você fez Tech for Tech. Se é a **operação**, você fez Tech for Business. As duas são legítimas — e nenhuma se defende com o argumento da outra.

TECH FOR TECH O CLIENTE É A PRÓPRIA ENGENHARIA

O AGENTE	O QUE ELE DEVOLVE
Revisor de pull request	risco apontado antes do merge · tempo de ciclo do PR
Triagem de incidente	hipótese e correlação em minutos · MTTA e MTTR
Guardião de custo de nuvem	recurso ocioso encontrado · US\$ devolvidos por mês
Migrador de dependência	biblioteca fora de suporte atualizada, com teste rodado
Documentador de API	OpenAPI gerado do código e sempre atual

TECH FOR BUSINESS O CLIENTE É A ÁREA DE NEGÓCIO

O AGENTE	O QUE ELE DEVOLVE
Deva · despesas	parecer com a regra aplicada · horas de conferência
Deva2 · editais	resposta com a cláusula citada · achado de auditoria
Deva3 · biometria	aprovar, revisar ou recusar · fricção no onboarding
Triagem de sinistro	dossiê montado e valor sugerido — nunca pago
Contestação de cobrança	investigação e parecer registrado · SLA ao cliente

os três agentes do curso são Tech for Business — de propósito

Tech for Tech é aprovado dentro de TI e o piloto começa na sexta-feira. **Tech for Business** precisa de dono na área, política escrita e alguém que assine a decisão: demora mais para começar, e é o que sobrevive ao corte.

E é por isso que o Tech for Tech quase nunca precisa da plataforma, e o Tech for Business quase sempre precisa: ele age **em nome de outra área**.

Memória e Skills, em versão prévia

Preview significa: bom para aprender e para prototipar, arriscado para prometer em contrato

FOUNDRY MEMORY (PREVIEW)

Três tipos, um ciclo

Perfil do usuário · resumo da conversa · procedural. O ciclo é sempre o mesmo: EXTRAÇÃO do que importa, CONSOLIDAÇÃO com o que já existia (usando um LLM para mesclar, deduplicar e resolver conflito) e RECUPERAÇÃO no momento certo.

Limites atuais: 100 escopos por repositório · 10 mil memórias por escopo · 1.000 requisições por minuto · retenção configurável — e SEM VNet.

FOUNDRY SKILLS (PREVIEW)

Procedimento salvo, não prompt

Segue a especificação aberta de agentskills.io: uma pasta com SKILL.md e frontmatter de name e description. O agente carrega só nome e descrição de todas as skills e decide por elas qual abrir.

A versão publicada é imutável (SkillVersion): mudou o procedimento, nasce versão nova.

É o SKILL.md da Aula 02, agora como recurso de plataforma.

A PERGUNTA DE GOVERNANÇA DA MEMÓRIA

Memória é dado pessoal armazenado. Quem apaga, e a pedido de quem? A resposta madura é usar os dois: Memory para conveniência, MEMORY.md para o que precisa de revisão humana e assinatura.

A ARMADILHA DAS SKILLS

DESCRIÇÃO VAGA = SKILL QUE NUNCA RODA. Como o agente escolhe lendo apenas nome e descrição, a descrição não é documentação: é a interface de seleção. Escreva o gatilho, não o resumo.

Os quatro motivadores — e nenhum deles é sobre o modelo

Quem diz «para fazer agente você precisa de uma plataforma de nuvem» está vendendo alguma coisa. Mas quando um destes aparece, a conversa muda

MOTIVADOR A

Dado e residência

A política diz que o dado não sai do tenant. Há exigência de rede privada, VNet ou residência regulatória.

Relevante em pagamentos, saúde e setor público. É requisito escrito, não preferência.

MOTIVADOR B

Identidade corporativa

O mais subestimado. Um agente que age em nome de um usuário precisa de identidade própria e permissões escopadas — não da sua chave de API com poder total.

Sem isso não existe resposta para «quem autorizou esta ação?».

MOTIVADOR C

Sandbox de execução

Se o agente executa código ou navega na web, precisa de isolamento real. Sem isso você roda código gerado por LLM em um lugar que importa.

MOTIVADOR D

Plano de controle

Rastreamento de cada chamada, avaliação de regressão, versionamento com rollback e custo por agente.

Vira obrigatório quando o agente é de operações — não seu.

A RECOMENDAÇÃO

Comece pela API direta. Migre quando um motivador aparecer — e ele quase sempre é conformidade ou identidade, nunca capacidade do modelo.

OS CUSTOS ESCONDIDOS

Latência da camada de abstração · defasagem de versão de modelo · acoplamento do plano de controle. O modelo você troca fácil; a orquestração, a memória e a observabilidade ficam no formato dela.

3

A decisão do modelo

Cinco famílias, cinco portas de entrada diferentes.

E a porta muda cobrança, crédito, cota e SLA — antes de qualquer conversa sobre qualidade.

Como os cinco entram no Foundry

A pergunta que decide o projeto não é «qual é melhor». É por qual porta ele entra

FAMÍLIA	PORTA DE ENTRADA	COMO É COBRADO	CRÉDITO AZURE	SLA E OPERAÇÃO	O QUE ISSO SIGNIFICA EM AULA
OpenAI GPT-5.6 Sol · Terra · Luna	vendido pela Azure	Medidor Azure na sua própria assinatura. Standard Global, Priority, Batch, PTU.	Sim	Hospedado e operado pela Azure, sob SLA Azure. 28 regiões.	É o caminho sem atrito. Funciona com Azure for Students.
DeepSeek V4 Pro · V4 Flash · V3.2 · R1	vendido pela Azure	Medidor Azure, implantação serverless. Global, DataZone e Regional.	Sim	Hospedado e operado pela Azure, sob SLA Azure.	Peso aberto com fatura de nuvem — e preço de nuvem (veja o próximo slide).
Anthropic Claude Opus 5 · Sonnet 5 · Haiku 4.5	parceiro · Marketplace	CCU — Claude Consumption Unit. Medido por hora, uma linha só na fatura.	Não. Mas decrementa o MACC.	Duas versões: na infra da Anthropic, ou ponta a ponta na Azure (GA).	Assinatura Student, Free, CSP ou patrocinada não compra do Marketplace.
Moonshot — Kimi K2.5 · K2.6 · K2.7 Code	parceiro · Marketplace	Por token, cobrado pelo Marketplace. Global Standard, 27 regiões.	Não	Publicador define preço, licença e regiões.	Peso aberto (MIT modificada) — a mesma restrição de assinatura vale.
Google — Gemini	não está no catálogo	—	—	Vive no Vertex AI / AI Studio, do outro lado da rua.	Do Google, o Foundry traz os pesos abertos (Gemma), não o Gemini.

Duas consequências práticas, nesta ordem: **1)** crédito de estudante e crédito pré-pago não compram modelo de Marketplace — o laboratório precisa de um modelo vendido pela Azure; **2)** o modelo que não está no catálogo entra como **ferramenta**, atrás de uma API sua — e aí você volta a carregar identidade, rede e auditoria no colo.

Catálogo e regras de cobrança conferidos no Microsoft Learn em 01/09/2026. Catálogo de nuvem muda sem aviso — revalide antes de reministrar.

Preço de referência dos cinco

Dólares por milhão de tokens, lista pública, 01/09/2026 — referência para o exercício, não para a reunião

MODELO	ENTRADA	SAÍDA	ENTRADA EM CACHE	CONTEXTO	OBSERVAÇÃO QUE MUDA A CONTA
GPT-5.6 Sol	5,00	30,00	0,40	1,05 M	Promoção até 21/11/2026 : 4,00 / 20,00.
GPT-5.6 Terra	2,00	12,00	0,20	1,05 M	O padrão de trabalho. É o modelo do Deva2.
GPT-5.6 Luna	0,20	1,20	0,02	1,05 M	25× mais barato que o Sol — e igual em classificação.
Claude Opus 5	5,00	25,00	0,1×	1 M	Contexto de 1 M sem sobretaxa . Lote: −50%.
Claude Sonnet 5	2,00	10,00	0,1×	1 M	Acerto de cache paga a si mesmo na primeira releitura.
Claude Haiku 4.5	1,00	5,00	0,1×	200 K	O barato da família — e o de cota mais folgada.
Kimi K2.6 no Foundry	0,95	4,00	—	262 K	Peso aberto, licença MIT modificada.
DeepSeek V4 Pro no Foundry	~1,96	~3,92	—	1 M	Direto na DeepSeek: 0,435 / 0,87 .
Gemini 3.1 Pro fora do Foundry	2,00	12,00	0,20	—	Preview no Vertex AI. Entra só como ferramenta.

A SOBRETAXA QUE NINGUÉM ORÇA

No GPT-5.6, requisição com **mais de 272 mil tokens de entrada** paga **o dobro na entrada e 1,5× na saída** — na requisição inteira.

PREÇO POR TOKEN NÃO É CUSTO

O que decide é **custo por execução**: a razão entrada/saída do seu caso muda o ranking — e às vezes inverte.

ONDE O DESCONTO MORA

Cache de prompt (0,1× a 0,02×), lote assíncrono (−50%) e **cota ciente de cache** no Claude. Desconto de arquitetura, não de negociação.

O mesmo modelo aberto, 4,5× mais caro

Um caso público de agosto de 2026 — e a resposta da Microsoft foi «está correto»

DIRETO NA DEEPSEEK

0,435 / 0,87

US\$ por 1M de tokens, entrada / saída. Acerto de cache: 0,003625.

→

NO MICROSOFT FOUNDRY

1,96 / 3,92

US\$ por 1M de tokens, entrada / saída. Sem medidor de entrada cacheada exposto.

O IMPACTO RELATADO

~4,5×

Cerca de 116 milhões de tokens por semana ≈ US\$ 1.176 por mês ao preço do Foundry. O mesmo peso, a mesma resposta.

POR QUE NÃO É BUG

O preço do Foundry embute o que você não compra da DeepSeek: infraestrutura gerenciada, conformidade, autenticação no **Entra ID** e RBAC, monitoramento no Gerenciamento de Custos, suporte e **compromisso de SLA**.

E o medidor de **entrada em cache** da DeepSeek não é exposto igual: a comparação linha a linha já nasce injusta.

A PERGUNTA CORRETA

Não é «por que estão me cobrando a mais?». É:

«eu preciso de identidade corporativa, rede privada, auditoria e SLA para esta carga?»

Se sim, você está pagando por algo. Se não, o endpoint do fornecedor resolve — e a diferença de 4,5× vira orçamento de outra coisa.

O QUE FAZER COM ISSO

- Compare **Global × Data Zone** antes de fechar
- Teste a SKU mais barata da família (**V4 Flash**) contra a sua suíte
- Se o volume é previsível, negocie preço comprometido
- E meça **custo por execução**, não preço por token

A lição generaliza para qualquer peso aberto em nuvem gerenciada: **você não está comprando o modelo, está comprando a operação dele**. Se a operação não é requisito, você está pagando por ela do mesmo jeito.

Claude no Foundry: CCU, MACC e a assinatura que não compra

O detalhe operacional que derruba um laboratório inteiro na primeira aula — e que não aparece na página do modelo

O QUE É A CCU

Uma unidade, não um medidor por modelo

Claude Consumption Unit: os tokens são medidos por modelo, aplicados às tarifas da Anthropic, descontados os contratos — e só então convertidos em CCU a um preço fixo.

Na fatura aparece uma linha só: CCU × preço. E ela é medida por hora, no Azure Marketplace.

ONDE O CUSTO APARECE

Três lugares, três granularidades

Gerenciamento de Custos: uma linha agregada de CCU — o que você paga.

Foundry → Monitorar: tokens por modelo, requisições e custo estimado — o que você opera.

Oferta privada no Marketplace: o preço contratado.

O QUE NÃO MUDA — NADA NO SEU CÓDIGO

Mesmo SDK, mesma API, mesmas tarifas por token, mesmos limites por chamada e a mesma relação de faturamento com a Microsoft.

A CCU é a embalagem da cobrança, não uma mudança de contrato técnico.

A ARMADILHA DE SALA

Assinatura Student, Visual Studio Enterprise, conta de crédito gratuito, CSP ou patrocinada por crédito NÃO compra oferta SaaS do Marketplace.

Como Claude e Kimi entram por ali, o laboratório precisa de um modelo VENDIDO PELA AZURE — GPT-5.6 ou DeepSeek — ou de assinatura paga.

E O QUE COMPENSA

O gasto em CCU decrementa o Microsoft Azure Consumption Commitment como qualquer consumo de Marketplace — em empresa com MACC, isso muda a conversa inteira.

E o limite de entrada por minuto só conta token NÃO cacheado: cache virou vazão.

A distância diminuiu — e a conta mudou de lugar

O roteiro dizia que os modelos abertos ganharam força. Ganharam. O que mudou é onde o custo aparece

O QUE MUDOU A FAVOR DO ABERTO

Eles entraram no catálogo gerenciado

Kimi K2.5, K2.6 e K2.7 Code aparecem no Foundry em Global Standard, com 27 regiões. DeepSeek V4 Pro e Flash são VENDIDOS PELA AZURE, com SLA e crédito.

Do Google, o que entra são os pesos abertos (Gemma), não o Gemini.

O QUE CONTINUA VALENDO

Peso aberto dá portabilidade

Você pode baixar e rodar na sua máquina, ou num sandbox, para testar sem enviar dado a lugar nenhum. É o argumento de soberania — e o único caminho para operação offline.

Mas rode em ambiente fechado: modelo aberto alucina igual.

O QUE A NUVEM COBRA POR CIMA

O preço não é o do fornecedor

DeepSeek V4 Pro custa ~4,5× mais no Foundry que no endpoint da própria DeepSeek. Não é bug: é infraestrutura gerenciada, Entra ID, RBAC, monitoramento, suporte e SLA.

Se você não precisa disso, está pagando por isso do mesmo jeito.

O QUE NÃO MUDOU

Nada disso é sobre inteligência

Um modelo barato com instrução bem escrita bate um modelo caro com instrução vaga. Trocar de modelo não conserta instrução mal escrita.

E instrução bem escrita faz um modelo barato parecer caro.

O EXPERIMENTO HONESTO DE SALA

Pegue o caso do seu agente. Rode a mesma suíte de avaliação em GPT-5.6 Luna, Kimi K2.6 e DeepSeek V4 Flash. Compare custo POR EXECUÇÃO e a taxa de acerto. Na maior parte das tarefas de classificação e extração, a diferença de qualidade não paga a diferença de preço — e é essa a descoberta que muda o orçamento do ano.

Comparar modelos sem se enganar

São quatro dimensões, não uma — e uma delas metade da turma lê ao contrário

DIMENSÃO	MÉTRICA E BENCHMARKS	LEITURA	O QUE ELA NÃO DIZ
Qualidade	Índice de qualidade (0 a 1), de MMLU-Pro, GPQA, MBPP+ e BigBench Hard.	maior é melhor	Nada sobre o seu domínio. Benchmark público é média de tarefas públicas.
Segurança	Taxa de sucesso de ataque (ASR), de HarmBench, Toxigen e WMDP.	menor é melhor	É a métrica que mais gente lê ao contrário no primeiro contato. Confirme a direção do eixo antes de decidir.
Desempenho	Latência P50/P90/P99, tokens por segundo, tempo até o primeiro token.	depende do caso	A média engana. Agente conversacional sofre no P90 ; processamento em lote não sente.
Custo	US\$ por execução do benchmark, com consumo real de tokens.	menor é melhor	Preço por token não é custo. A razão entrada/saída do seu caso muda o ranking — às vezes inverte .

1 · COMECE PELO CENÁRIO

Raciocínio, código, matemática, perguntas e respostas, fundamentação ou segurança. O placar do **seu** caso vale mais que o índice geral.

2 · EQUILIBRE COM DESEMPENHO

Latência alta mata experiência conversacional e não incomoda em lote. Olhe o P90, não a média.

3 · OLHE A SEGURANÇA

Se o agente fala com cliente ou com o público, o ASR deixa de ser curiosidade e vira requisito.

4 · CUSTO POR EXECUÇÃO

Compare o custo de resolver a tarefa inteira, não o preço da unidade.

5 · AVALIE NOS SEUS DADOS

Benchmark é o currículo. Avaliação no seu conjunto é a entrevista. De 20 a 50 casos vindos de falhas reais, metade negativos.

Sem suíte de avaliação você não sabe se trocar o modelo melhorou ou piorou — e não consegue defender a escolha em comitê. **Escreva a suíte antes de escolher o modelo.**

4

Onde nasce, onde aparece, o que freia

O agente prompt é de graça. O que ele usa, não.
E o orçamento do Azure avisa — ele não freia.

Onde o Foundry cobra

Oito medidores — e o único que costuma dominar a conta é o primeiro

	MEDIDOR	PREÇO DE REFERÊNCIA	O QUE LIGA ESSE MEDIDOR SEM VOCÊ NOTAR
1	Tokens do modelo	varia por modelo	Nada: é o uso normal. Costuma ser ~98% da conta de um agente <code>prompt</code> .
2	Agente <code>prompt</code> ou <code>workflow</code>	US\$ 0,00	Sem cobrança pelo agente em si, nem pela orquestração.
3	Agente <code>hosted</code> — vCPU	~US\$ 0,0864 / vCPU-hora	Réplica mínima maior que zero: o contêiner fica faturando fora do horário.
4	Agente <code>hosted</code> — memória	~US\$ 0,0090 / GiB-hora	Idem. Escala a zero e tempo de ociosidade curto resolvem.
5	Conhecimento — vetorial	~US\$ 0,10 / GB-dia	O primeiro 1 GB é gratuito. Acima disso é custo diário permanente : subir um PDF grande «por garantia» cobra todo dia.
6	Interpretador de código	~US\$ 0,03 / sessão	A sessão dura enquanto o thread estiver ativo. Ferramenta ligada «por via das dúvidas» cobra em toda conversa.
7	Pesquisa na web (grounding)	~US\$ 35,00 / mil transações	Cobrado pelo recurso de grounding, não pelo agente. É o mais fácil de estourar com um agente que pesquisa a cada volta.
8	Application Insights	~US\$ 2,30 / GB ingerido	O interruptor «Configure os recursos recomendados» vem LIGADO e provisiona o recurso em todo projeto novo.

Fora desta tabela, e de propósito: **ajuste fino** e **capacidade provisionada (PTU)** cobram **hospedagem por hora, com ou sem uso**. Não são custo variável — são custo fixo mensal, e entram em outra linha do orçamento.

Valores de referência coletados em 31/08/2026 para exercício de sala. Confirme região e contrato no Calculador de Preços do Azure.

A conta de um agente real

Deva2 — Analista de Editais · agente prompt · GPT-5.6 Terra · 200 consultas por dia, 22 dias

COMPONENTE	QUANTIDADE	PREÇO UNIT.	CUSTO NO MÊS
Tokens de entrada 8.000 por consulta	35,2 M	2,00	US\$ 70,40
Tokens de saída 700 por consulta	3,08 M	12,00	US\$ 36,96
Agente prompt	1	0,00	US\$ 0,00
Conhecimento (0,5 GB)	no 1 GB grátis	0,10	US\$ 0,00
Interpretador de código	0 sessões	0,03	US\$ 0,00
Pesquisa na web	desligada	35,00	US\$ 0,00
Application Insights	1 GB	2,30	US\$ 2,30
TOTAL MENSAL			US\$ 109,66

A PARTICIPAÇÃO DO MODELO

98%

do total. Se este número vier **baixo** na sua conta, o problema de custo **não é o modelo** — é ferramenta ou infraestrutura ligada sem uso.

CUSTO POR CONSULTA

US\$ 0,025

É este o número que se leva para a reunião — não o total. Ele compara com o custo do processo manual que o agente substitui.

O experimento de sala: troque só o modelo, de Terra para Luna — o total cai de US\$ 109,66 para **US\$ 13,04**. Agora rode a suíte de avaliação e responda: a qualidade caiu? · E lembre: estimativa serve para **decidir**; fatura serve para **pagar**.

Onde medir — e por que os três números não batem

Não é defeito: é latência de ingestão. Quem confunde os três promete número errado para a diretoria

1 · **FOUNDRY** → **OPERAR** → **VISÃO GERAL**

Quanto está custando AGORA

Quase em tempo real. Serve para **operar**: decidir hoje se desliga alguma coisa.

2 · **FOUNDRY** → **AGENTE** → **MONITORAR**

QUAL agente está caro, e por quê

Quase em tempo real, por agente. É onde você mede os tokens reais por consulta e corrige a estimativa da calculadora.

3 · **AZURE** → **GERENCIAMENTO DE CUSTOS**

O que você PAGA

Algumas horas de atraso. Agrupe por **Medidor** e por **Recurso**; filtre pela etiqueta **project**, aplicada sozinha a todo projeto — e você tem rateio sem planilha.

O FREIO REAL NÃO É O ORÇAMENTO

O Azure OpenAI **não tem limite rígido de gasto**. O orçamento não para nada: ele manda e-mail. O que efetivamente interrompe o gasto é a **cota** — tokens por minuto por implantação.

E a cota varia por porta de entrada. No Claude dentro do Foundry, em pay-as-you-go, ela é **compartilhada pela assinatura inteira**:

MODELO	RPM	ITPM	OTPM
Opus 5 · Opus 4.8 · Sonnet 5	40	40.000	8.000
Sonnet 4.6 · 4.5 · Haiku 4.5	80	80.000	16.000

O ITPM é **ciente de cache**: só entrada não cacheada conta para o limite. Cache deixou de ser só economia e virou **vazão**.

O QUE LIGAR ANTES DE OPERAR

- Orçamento com alerta em **50%, 80% e 100%**
- Limite de **cota TPM** por implantação — o único freio real
- Retenção definida no Application Insights
- Réplica mínima **0** em tudo que for **hosted**
- Implantações revisadas ao fim de cada sprint
- Em laboratório: **um comando que apaga tudo**

Sete formas de estourar a fatura sem aviso vermelho

As quatro primeiras cobram com o projeto parado — e é por isso que elas passam despercebidas até o fechamento do mês

1 · AJUSTE FINO

Cobra hospedagem POR HORA, mesmo sem uso. Um deployment esquecido depois de um teste continua faturando dia e noite.

→ Liste implantações no fim de cada sprint e exclua o que não está em uso.

2 · CAPACIDADE PROVISIONADA (PTU)

Reserva e cobra por hora. Ligado ou não, você paga a capacidade reservada.

→ Só com volume previsível e medido. Nunca em laboratório de aula.

3 · MODELO DE MARKETPLACE

Não aceita crédito pré-pago. Aparece como recurso «Global» e fura o orçamento que você achava que existia.

→ Confira a origem do modelo ANTES de implantar.

4 · ORÇAMENTO DO AZURE

Ele não para; ele manda e-mail. O Azure OpenAI não tem limite rígido de gasto.

→ Alerta em 50%, 80% e 100% mais limite de cota TPM por implantação. A cota é o único freio real.

5 · «RECURSOS RECOMENDADOS»

O interruptor vem LIGADO e provisiona um Application Insights que cobra por volume ingerido em todo projeto novo.

→ Desligue em projeto de estudo. Em produção, ligue de propósito e defina retenção.

6 · RASTREAMENTO

Guarda o que o usuário digitou: os traços podem conter dado pessoal e Conteúdo do Cliente, e quem tem leitura no App Insights vê. É custo e é LGPD.

→ Retenção, papel de leitura restrito e base legal registrada.

7 · CONHECIMENTO ALÉM DE 1 GB

O primeiro 1 GB de armazenamento vetorial é gratuito; o resto é cobrado POR DIA, para sempre.

→ Suba só o que o agente precisa citar. Arquivo grande «por garantia» é assinatura mensal.

E O HÁBITO QUE RESOLVE

Todo laboratório nasce com um grupo de recursos próprio e morre com um comando:

`az group delete --name RG --yes`

→ E lembre: `az vm stop` cobra; só `deallocate` libera.

5

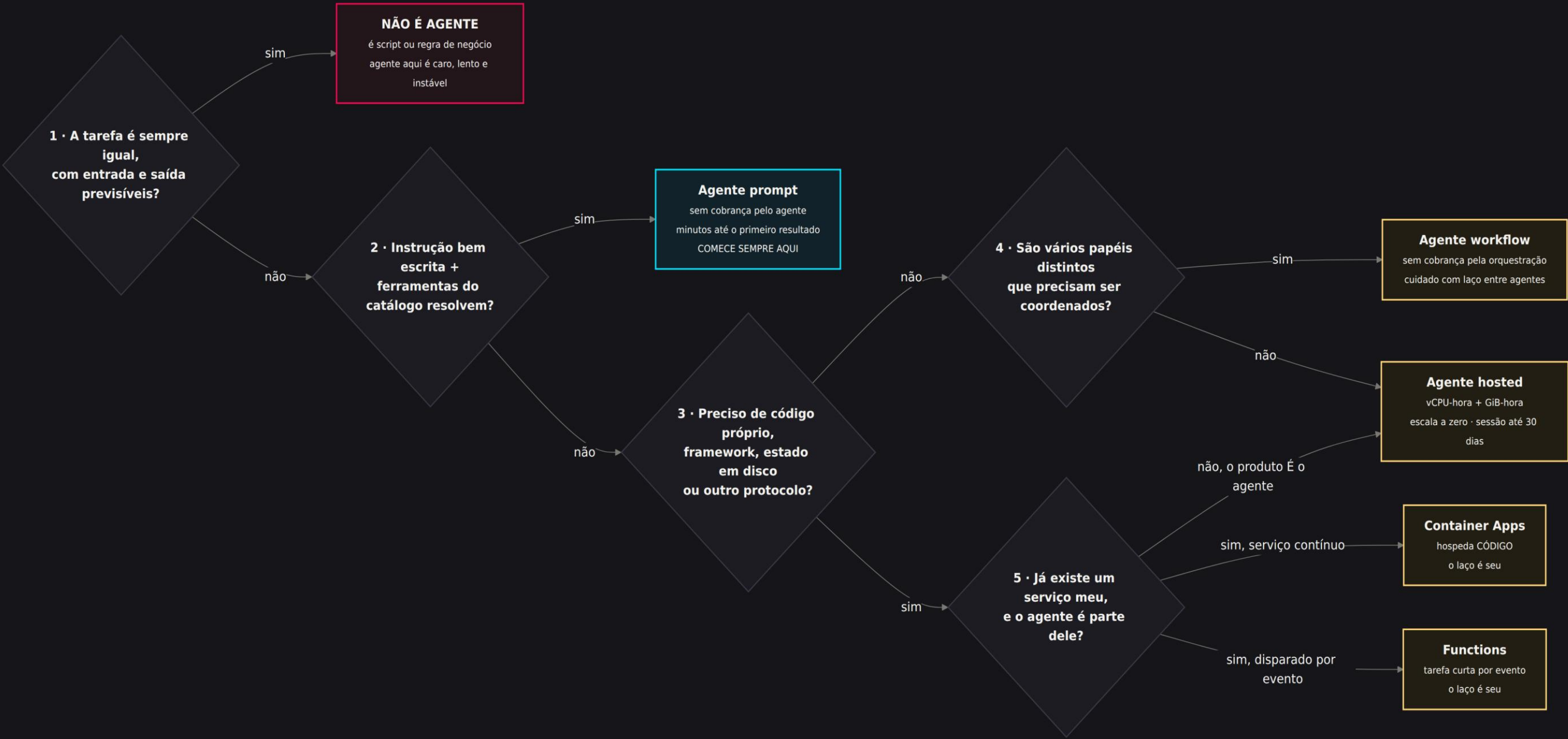
Quando usar cada um — e quando alternar

Quatro perguntas, nesta ordem. A primeira é a que mais economiza dinheiro, porque a resposta certa dela costuma ser «isto não é um agente».

As perguntas, na ordem que economiza

Cada nó só existe porque a resposta do anterior fechou uma porta

Árvore de decisão — que recurso usar, e quando alternar



A PERGUNTA 1 É A QUE ECONOMIZA

Se a tarefa é sempre igual, com entrada e saída previsíveis, NÃO é agente. É script ou regra de negócio. Agente ali é caro, lento e instável — e ninguém consegue explicar por que a resposta mudou.

A REGRA QUE ECONOMIZA SEMESTRE

Ninguém começa em hosted. Você começa em prompt, descobre o que falta e PROMOVE quando o limite aparecer. Escolher a opção complexa primeiro é o erro mais caro e mais comum.

QUEM É DONO DO LAÇO

Container Apps e Functions hospedam CÓDIGO. O Foundry hospeda um AGENTE — com identidade, rastreamento, avaliação e conta separada.

Fonte: docs/diagramas/02-arvore-decisao.mmd

Foundry × Container Apps × Functions

A pergunta não é qual é melhor. É quem executa o laço do agente

CRITÉRIO	FOUNDRY AGENT SERVICE	AZURE CONTAINER APPS	AZURE FUNCTIONS
Quem executa o laço	O serviço. Você descreve o agente.	Você. O laço está no seu código.	Você, em execuções curtas por evento.
Ferramentas, memória e traço	Prontos: catálogo, Memória, Rastreamentos, avaliações.	Você monta e mantém tudo.	Você monta e mantém tudo.
Identidade por agente	Sim, principal próprio no Entra ID.	Por aplicação, não por agente.	Por aplicação.
Escala a zero	Sim, nos agentes hospedados.	Sim.	Sim, no plano de consumo.
Duração máxima	Sessão viva por até 30 dias.	Sem limite prático (serviço contínuo).	Minutos no consumo; mais no Premium.
Governança e auditoria	Nativas: traço, avaliação, etiqueta project.	Você instrumenta.	Você instrumenta.
Escolha quando	O produto É o agente e a empresa precisa auditar.	Já existe um serviço seu e o agente é parte dele.	Tarefa curta disparada por evento.
NÃO escolha quando	Você precisa de controle total do laço e de dependências exóticas.	Você quer governança pronta e não tem equipe para construí-la.	A tarefa é longa, conversacional ou mantém estado.

Container Apps e Functions hospedam código. O Foundry hospeda um agente. Se você vai reconstruir identidade, traço e avaliação à mão, já está pagando pelo Foundry sem usar o Foundry.

Quando trocar — e para onde

Troca de modelo é configuração; troca de plataforma é obra. Cada linha tem um gatilho observável

O SINTOMA QUE VOCÊ OBSERVA	O GATILHO, EM NÚMERO	A AÇÃO	O QUE NÃO RESOLVE
A conta cresceu e a qualidade não mudou	Modelo > 90% do total e a suíte não distingue os dois modelos	Desça de faixa: Sol→Terra→Luna, Opus→Sonnet→Haiku. Rode a suíte antes.	Encurtar o prompt. O custo está no volume, não no texto.
A experiência conversacional ficou lenta	Latência P90 acima do aceito — média engana	Modelo mais rápido da mesma família, ou tirar um passo do laço.	Aumentar a cota. Cota é vazão, não latência.
Erros 429 em horário de pico	Cota por implantação: TPM ; no Claude, 40 RPM / 40k ITPM por assinatura	Pedir aumento de cota, distribuir entre implantações, ou cache ar — no Claude, só entrada não cacheada conta.	Trocar de modelo. O limite é da implantação, não da inteligência.
O agente fala com cliente ou com o público	ASR — taxa de sucesso de ataque; menor é melhor	Escolher pela dimensão de segurança do placar, não pela qualidade.	Escrever «não caia em golpe» na instrução.
A resposta melhora com contexto gigante	Entrada acima de 272 mil tokens no GPT-5.6	Modelo sem sobretaxa de contexto longo, ou recuperar menos e melhor.	Pagar e seguir. A sobretaxa dobra a entrada da requisição inteira.
O dado não pode sair do tenant	Política escrita, não preferência	Troca de plataforma: Data Zone, rede privada, modelo hospedado na Azure ponta a ponta.	Modelo melhor. Este eixo não é sobre capacidade.
Ninguém sabe qual agente gastou o quê	Uma chave para tudo; logs indistinguíveis	Troca de plataforma: identidade por agente e etiqueta project .	Planilha de rateio. Ela envelhece na semana seguinte.
A instrução ficou cheia de gambiarra	Três remendos na instrução para simular uma função	Promover de prompt para hosted .	Um quarto remendo.

A regra que atravessa a tabela: **comece pelo barato e suba quando falhar**, com a falha definida antes por uma suíte de avaliação. Começar pelo melhor «por garantia» é o erro de custo nº 1 — e ele nunca aparece com aviso vermelho na tela.

O roteador de modelo

O modelo é parâmetro, não destino — e o nome dele envelhece em meses

```
ROTEADOR = {
    "classificar": "gpt-5.6-luna",    # alto volume, decisão simples
    "explicar":    "gpt-5.6-terra",   # texto para humano
    "investigar":  "gpt-5.6-sol",     # exceção difícil, poucas vezes por dia
}

def cerebro(tarefa: str) -> str:
    return ROTEADOR.get(tarefa, "gpt-5.6-luna")    # padrão: o barato
```

COMECE PELO BARATO

E suba só quando a suíte de avaliação falhar. Começar pelo melhor «por garantia» é o erro de custo nº 1 em projeto de agente.

UMA CONSTANTE SÓ

Nome de modelo espalhado pelo código faz de cada troca uma refatoração. Uma constante, e a troca vira uma linha.

E A VERSÃO QUE A PLATAFORMA JÁ OFERECE

Model router

O Foundry tem um roteador nativo que casa cada requisição com o modelo adequado, ao lado de cache de prompt e gestão de catálogo de ferramentas. Vale considerar quando o tráfego é heterogêneo e você não quer manter a tabela na mão.

Mas mantenha a tabela explícita enquanto estiver aprendendo: o roteador automático esconde justamente a decisão que este bloco quer ensinar.

E lembre da ordem certa das alavancas de contexto, antes de trocar qualquer coisa:

- 1 · Compactação
- 2 · Notas estruturadas
- 3 · Subagentes — a mais cara das três

Quem inverte essa ordem paga muitas vezes mais para resolver um problema que uma boa compactação resolveria.

O eixo que decide a autonomia

Não é «sugere ou executa». É reversibilidade e verificabilidade — e essas duas perguntas cabem em uma reunião

	VERIFICÁVEL POR CÓDIGO	SÓ UM HUMANO CONFERE
Reversível	AUTONOMIA TOTAL consultar o edital · buscar a política · simular um cálculo · detectar o rosto e devolver a confiança	AUTONOMIA COM LOG E AMOSTRAGEM rascunhar o parecer · sugerir a classificação da despesa · propor uma linha de memória
Irreversível	AUTONOMIA COM CONFIRMAÇÃO gravar registro idempotente · abrir o protocolo de exceção · publicar uma versão nova do agente	PORTÃO HUMANO OBRIGATÓRIO aprovar o reembolso · lançar no financeiro · comunicar o cliente · aprovar a própria memória · alterar a alçada

COMO ISSO VIRA CÓDIGO

O quadrante de baixo à direita **não vira instrução**. Vira operação que não existe na especificação que o agente carrega. No Lab2, a API tem **14 operações** e o agente recebe **5** — nenhuma delas aprova coisa alguma.

O QUE O MERCADO FEZ

Quem foi a produção executa só o reversível e verificável — bloquear cartão, criar caso, transferir entre contas próprias — e **para antes da decisão irreversível**: pagar sinistro, publicar regra de fraude, decidir crédito.

AS TRÊS CAUSAS DE EXCESSO

Funcionalidade em excesso: a ferramenta faz mais do que precisa. **Permissão** em excesso: direito a mais nos sistemas de baixo. **Autonomia** em excesso: ação de alto impacto sem humano.

Instrução é intenção. Permissão é controle. **Em agentes, o que você não quer que aconteça, você não expõe.**

6

Deva contínuo

Memória revisável, fila de documentos e o gatilho que faz o agente começar sozinho.

Repositório: github.com/IsaiasBritto/aie-cloud — aulas/05-foundry-agents/Lab2

Duas perguntas para a turma:

Nenhuma delas se responde com um slide. As duas se respondem com um projeto

PERGUNTA 1

«Ele lê nota fiscal, mas não parece um agente contínuo.»

Como incrementá-lo para ter esse comportamento?

PERGUNTA 2

«Como o aluno abre o MEMORY.md para saber que ele está aprendendo?»

Se não dá para ver, não dá para ensinar — nem para auditar.

A DEMONSTRAÇÃO DE 30 SEGUNDOS — COMECE MOSTRANDO O PROBLEMA

- 1 · Abra o agente no Playground e corrija-o: «estacionamento em aeroporto é viagem aérea».
- 2 · Ele concorda, educadamente.
- 3 · Abra uma conversa NOVA e faça a mesma pergunta.
- 4 · Ele erra igual.

«Vocês acabaram de ensinar uma coisa a ele. Onde foi parar?» — e deixe o silêncio durar.

A PERGUNTA QUE ABRE O BLOCO — QUEM VERIFICA A CONDIÇÃO?

Leia em voz alta a seção 7 do AGENTS.md v1.3:

«Escreva em MEMORY.md somente quando o auditor humano corrigir você.»

E pergunte: quem verifica que essa condição foi satisfeita?

Espere. A resposta é: o próprio agente.

O QUE O PROJETO ENTREGA

Serviço de Continuidade em FastAPI, com memória revisável e fila.

Tela em Streamlit com quatro abas: Painel · Memória · Propostas · Exceções.

Gatilho por evento no Blob.

Ferramenta OpenAPI ligada ao agente — 5 das 14 operações.

A escada da continuidade

Continuidade não é um interruptor. São cinco degraus, e cada um exige alguma coisa construída

	O DEGRAU	O QUE MUDA	O QUE PRECISA EXISTIR	QUEM JÁ RESOLVE
0	Pergunta e resposta	Nada persiste.	—	qualquer chat
1	Sessão com contexto	Lembra dentro da conversa.	thread gerenciado	o Foundry já faz
2	Memória entre sessões	Lembra amanhã .	<code>MEMORY.md</code> em armazenamento durável + a regra de ler no início	Foundry Memory (preview) ou o arquivo
3	Iniciativa	Começa sozinho . É o degrau que muda a percepção da turma.	gatilho por evento : Event Grid → Logic App, ou sondagem em sala	ninguém — você constrói
4	Fila de exceções	Chama gente só quando precisa . É o que separa automação de brinquedo.	máquina de estados explícita, com a transição proibida escrita em código	ninguém — você constrói

O QUE DIZER NO NÍVEL 3

Enquanto alguém digita a pergunta, ele parece um chat com esteroides. Quando o aluno larga um PDF numa pasta e o agente **acorda sozinho**, ele vira processo.

O filtro que ninguém lembra: `subjectBeginsWith` no contêiner de entrada — senão o `MEMORY.md` que o serviço escreve dispara o fluxo.

O NÚMERO DO NÍVEL 4

Não é quantos documentos ele processou. É **quantos ele devolveu**. Um agente que devolve 40 itens para revisão não economizou nada.

Se mais de **30%** param em exceção, o problema é a regra, não o modelo.

O MODO DE FALHA DO CONTÍNUO

Exceção **não volta** para o agente, mesmo que ele ache que sabe resolver:

```
`if estado is EXCECAO and por == 'deva':
    raise TransicaoProibida`
```

Ele não gasta muito por volta; gasta por não parar nunca.

O agente propõe; ele não decide

Dois arquivos, e uma linha atravessando de um para o outro — com o nome de quem assinou



MEMORIA-PENDENTE.MD

O que o agente QUER aprender. Não influencia uma única decisão dele. É fila de aprendizado, não memória — e fila que ninguém revisa é dívida: bate o teto de 50 e ele para de propor.

MEMORY.MD

O que ele PODE usar. A origem da linha é o NOME DO AUDITOR que aprovou, nunca «deva». Um agente que aprende sem que ninguém assine o aprendizado é um agente indefensável numa auditoria.

OS DOIS CAMINHOS DE RECUSA

Padrão de manipulação vira incidente de segurança; tentativa de alterar alçada vai para a Controladoria. E seja honesto: o filtro de texto é rede de proteção didática. Quem segura de verdade é a falta do cabeçalho X-Auditor.

Fonte: docs/diagramas/03-fluxo-da-memoria.mmd · o filtro corrigido: mencionar um limite é trabalho normal de auditor; MUDAR o limite é outra coisa. O filtro precisa das duas peças.

O que cada peça do Lab2 prova

Módulo → arquivo → conceito. Se o aluno não consegue apontar o arquivo, ele não entendeu o conceito

O QUE O ALUNO FAZ	ONDE ISSO VIVE NO REPOSITÓRIO	O CONCEITO QUE FICA PROVADO
Sobe o serviço e a tela	<code>api/principal.py</code> · <code>web/aplicacao.py</code>	Memória é um arquivo , com dono, permissão e retenção — não uma caixa-preta.
Roda o gatilho com <code>--semear</code>	<code>gatilho/disparador.py</code>	Nível 3: ninguém digitou nada. Um arquivo chegou e o processo começou.
Roda o ciclo três vezes	<code>gatilho/ciclo_do_agente.py</code>	O estado é que diz ao agente o que fazer agora, sem ninguém programar a ordem.
Pede ao agente para aprovar a própria proposta	<code>agente/openapi-agente.json</code>	403 · a fronteira . A API tem 14 operações; a especificação do agente tem 5, e nenhuma aprova.
Aprova pela aba Propostas com o seu nome	<code>api/servicos/memoria.py</code>	A linha do <code>MEMORY.md</code> leva o nome do auditor , nunca «deva». Aprendizado sem assinatura é indefensável.
Tenta liberar a exceção pelo agente	<code>api/servicos/fila.py</code>	409 . Exceção não volta para o agente. A transição proibida é código , não recomendação.
Pede memória com uma frase de injeção	<code>PADROES_SUSPEITOS</code> em <code>memoria.py</code>	Texto lido de documento é dado, nunca instrução — e quem segura é a falta do cabeçalho <code>X-Auditor</code> .
Roda os testes	<code>api/testes/</code> — 22 testes, sem rede	A garantia é testada , não prometida.

DOIS MINUTOS, SEM AZURE

```
python gatilho/disparador.py --semear
python gatilho/ciclo_do_agente.py --uma-volta
(três vezes: extraído · duplicado · exceção)
```

O ENTREGÁVEL — TRÊS COISAS NA MESMA TELA

- a proposta na fila, com a **evidência**
- o **403** que o agente recebeu ao tentar aprovar
- a linha já no `MEMORY.md`, **assinada**

CUSTO

Local: **US\$ 0,00**. Na Azure: **menos de US\$ 1,00** por aluno.

Encerramento:

```
bash infra/99-remove-tudo.sh
```

O roteiro, bloco a bloco

Entra depois do laboratório do Deva, quando a turma já viu o agente ler nota fiscal e responder

TEMPO	BLOCO	O QUE ACONTECE
4 min	A pergunta	A demonstração dos 30 segundos: ele esquece.
6 min	Os cinco níveis	A escada, com o degrau que muda a percepção.
8 min	Memória revisável	Proposta × escrita, com a evidência do próprio laboratório.
10 min	Mão na massa	Ciclo local rodando: gatilho, fila, exceção. Três coisas para apontar na tela, nesta ordem: o duplicado, a exceção, e o ciclo que roda de novo sem fazer nada.
8 min	A aprovação ao vivo	403 ao tentar aprovar a própria proposta. Depois o auditor assina, e a linha atravessa para o MEMORY.md.
5 min	A tentativa de injeção	O filtro disparando na frente da turma — e a confissão de que ele não é segurança.
4 min	Fechamento	Instrução × permissão.

SE O TEMPO APERTAR

Faça a aprovação ao vivo e a tentativa de injeção. São os dois blocos que os alunos comentam com outras pessoas depois — e os dois que provam a tese do módulo sem precisar de slide.

AS PERGUNTAS QUE A TURMA FAZ

- «Não dá para usar o Foundry Memory e pular isso?» — Dá, e em produção costuma valer. Mas é caixa-preta: o aluno não vê o texto, e o texto é o conteúdo. A resposta madura usa os dois.
- «Isso não deixa o agente lento?» — Duas chamadas HTTP por volta. O gargalo é o modelo.
- «Quem apaga uma memória errada?» — Hoje, ninguém. De propósito.

OS QUATRO EXERCÍCIOS DE DEPOIS

- 1 · Esquecimento — POST /memoria/{id}/arquivar. Uma regra de 12 meses sem uso continua valendo? Quem decide?
- 2 · Rastreabilidade — cada decisão registra QUAL regra aplicou. Ao arquivar a regra, liste o que dependeu dela.
- 3 · Taxa de devolução — meça o % que para em exceção. Passou de 30%, o problema é a regra. Prove.
- 4 · Segunda opinião — um segundo agente critica a proposta. Ajuda, ou vira teatro?

As seis lições

Uma por bloco — e cada uma é cobrada em pelo menos uma decisão real de projeto

LIÇÃO 1

Um agente é um laço, não um framework

Doze linhas contêm o sistema inteiro. Orçamento de passos, erro como observação e guardrail em código separam demonstração de produção.

LIÇÃO 2

A plataforma não deixa o agente mais inteligente

Deixa ele auditável, orçado e revogável. É por isso que a decisão de nuvem é sobre identidade, rede, segredo, escala e auditoria — nunca sobre capacidade.

LIÇÃO 3

A porta de entrada do modelo decide mais que o modelo

Vendido pela Azure ou parceiro no Marketplace muda cobrança, crédito, cota e SLA. Descobrir isso na hora do laboratório custa a aula inteira.

LIÇÃO 4

O orçamento avisa; a cota freia

O Azure OpenAI não tem limite rígido de gasto. O único freio que interrompe é o limite de tokens por minuto por implantação — e cache virou vazão, não só economia.

LIÇÃO 5

Comece no barato e no simples, e promova por gatilho

prompt antes de hosted. Luna antes de Sol. E a promoção acontece quando um limite observável aparece — não quando alguém acha que o caso é complexo.

LIÇÃO 6

Instrução é intenção; permissão é controle

O que impede o agente de aprovar a própria memória não é a frase no AGENTS.md. É a operação não existir na especificação que ele carrega.

O QUE FICA

O Foundry não deixa o seu agente mais inteligente.

Ele deixa o seu agente auditável, orçado e revogável.

E o modelo, que é a peça que todo mundo passa mais tempo escolhendo, é a única que você troca com uma linha — desde que tenha escrito a suíte de avaliação antes.

Aula 02 · Deva

os arquivos do agente

Aula 03 · Deva2

a plataforma

Aula 05 · Deva3

o código próprio

Lab2 · Deva contínuo

a fronteira

PROF. ISAIAS S. BRITTO · FIAP · MBA AI ENGINEERING & MULTI-AGENTS